Sophos Firewall Site-to-Site VPN Lab on AWS

Complete build guide from scratch: every step explained, interview follow-up questions, and STAR stories

Author	Jeevanandh Ravi
Platform	AWS us-east-1 · Sophos Firewall SFOS 22.0.2 (PAYG) · Ubuntu 24.04 · Terraform 1.14
Repository	github.com/Jeevanandh32/sophos-firewall-site-to-site-vpn-lab
Built	October 2026

Contents

1. What the lab is, and why it's built this way
2. Architecture and address plan
3. Prerequisites
4. Phase 1: Infrastructure as code (Terraform, line by line)
5. Phase 2: Deploy
6. Phase 3: First login
7. Phase 4: Firewall rules and NAT
8. Phase 5: Web filtering
9. Phase 6: Site-to-site IPsec VPN
10. Phase 7: Validation (test script, line by line)
11. Phase 8: Backup and restore
12. Phase 9: Teardown and cost control
13. Problems hit during the real build, and the fixes
14. Interview follow-up questions and answers
15. STAR stories
16. Resume entry

1. What the lab is, and why it's built this way

The lab simulates two company offices (Site A and Site B). Each office has its own network, protected by a Sophos Firewall, and the two offices are joined by an encrypted IPsec tunnel across the internet. Each office has one Ubuntu machine that acts as a user's computer.

Every firewall skill an entry-level network or security role asks for shows up somewhere in the lab:

- **Zones and interfaces:** which port is trusted (LAN) and which is untrusted (WAN).
- **NAT:** how private 10.x addresses reach the internet through one public IP.
- **Firewall rules:** ordered allow and deny rules, with logging.
- **Web filtering:** blocking a website category by policy.
- **Site-to-site VPN:** IKEv2 IPsec between two firewalls, through NAT.
- **Verification:** proving each rule works with tcpdump, ping, curl and the firewall logs.
- **Change management:** a configuration backup, a change, and a restore.

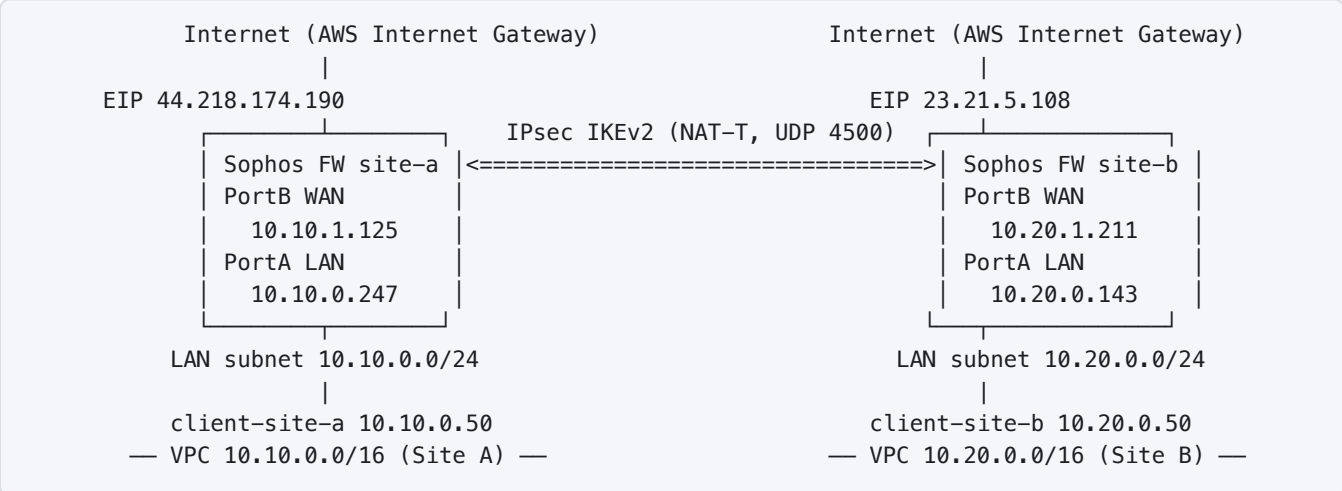
Why AWS and not a local VM?

The development laptop is an Apple Silicon (ARM) Mac. Sophos Firewall only ships for x86-64, and VMware Fusion and VirtualBox on Apple Silicon can only run ARM guests. AWS EC2 runs x86-64, and Sophos publishes an official image on AWS Marketplace, so the lab runs there.

Why Terraform?

Clicking through the AWS console is slow and hard to repeat. With Terraform, the whole network (two VPCs, subnets, routes, security groups, firewalls and clients) is described in code. It's created with one command, destroyed with one command, and the code is version-controlled on GitHub.

2. Architecture and address plan



Item	Site A	Site B	Why

VPC	10.10.0.0/16	10.20.0.0/16	The ranges must not overlap, or the VPN can't route between them.
LAN subnet (private)	10.10.0.0/24	10.20.0.0/24	No route to the internet except through the firewall.
WAN subnet (public)	10.10.1.0/24	10.20.1.0/24	Has a route to the Internet Gateway.
Firewall LAN (PortA)	10.10.0.247	10.20.0.143	Default gateway for the clients (through the AWS route table).
Firewall WAN (PortB)	10.10.1.125	10.20.1.211	Private address. AWS maps it 1:1 to the Elastic IP.
Elastic IP	44.218.174.190	23.21.5.108	Static public IP, used as the VPN peer address and IKE ID.
Client	10.10.0.50	10.20.0.50	Fixed IP so the tests can target the remote client.
Instance types	Firewall t3.medium (2 vCPU / 4 GB, the smallest Sophos allows) · client t3.micro		Lowest cost.

How a packet flows: client-site-a to client-site-b

1. The client sends to 10.20.0.50. Its subnet route table says `0.0.0.0/0 → firewall A's LAN interface`.
2. Firewall A matches the VPN firewall rule (LAN → VPN). The packet matches the IPsec policy 10.10.0.0/24 ↔ 10.20.0.0/24, so it gets encrypted with ESP.
3. The ESP packet is wrapped in UDP 4500 (NAT-T) and sent from 10.10.1.125 to 23.21.5.108. AWS rewrites the source to 44.218.174.190.
4. AWS delivers the packet to firewall B's 10.20.1.211. B decrypts it, checks its VPN → LAN rule, and forwards it out PortA to 10.20.0.50.
5. The client sees the source 10.10.0.50, because no NAT is applied inside the tunnel. The reply goes back the same way.

3. Prerequisites

Need	How	Why
AWS account and CLI	`aws configure`, then check with `aws sts get-caller-identity`	Terraform uses these credentials.
Terraform ≥ 1.5	`brew install terraform`	Runs the infrastructure code.
Marketplace subscription	Subscribe to "Sophos Firewall Standalone PAYG" and accept the terms	Without it, EC2 refuses to launch the image (`OptInRequired`).

Your public IP	`curl https://checkip.amazonaws.com`	Admin access is locked to this one /32.
GitHub CLI (optional)	`gh auth login`	Creates and pushes the repo.

4. Phase 1: Infrastructure as code (line by line)

The project layout:

```
Firewall/
├── terraform/
│   ├── versions.tf                  providers and versions
│   ├── variables.tf                 inputs (region, admin IP, sizes)
│   ├── main.tf                      all resources
│   ├── outputs.tf                   values printed after deploy
│   ├── sophos-standalone-payg.yaml  Sophos's official CloudFormation template
│   └── terraform.tfvars             your admin IP (git-ignored)
├── scripts/client-tests.sh          PASS/FAIL validation
├── RUNBOOK.md  README.md  .gitignore
```

4.1 versions.tf

Code	What it does and why
`terraform { required_version = ">= 1.5"`	Refuses to run on older Terraform, which lacks features this code uses.
`required_providers { aws = { source = "hashicorp/aws", version = ">= 5.60" }`	The AWS provider is the plugin that calls the AWS APIs. 5.60+ is needed for `aws_ec2_instance_connect_endpoint` and `aws_vpc_security_group_ingress_rule`.
`random = { source = "hashicorp/random" }`	Generates the admin passwords and a random suffix for names.
`tls = { source = "hashicorp/tls" }`	Generates the SSH key pair locally, so no key has to be created by hand.
`local = { source = "hashicorp/local" }`	Writes the private key to `.keys/sophos-lab.pem`.
`provider "aws" { region = var.region`	Sends every API call to us-east-1, or whatever region the variable says.
`default_tags { tags = { Project = "sophos-s2s-lab" } }`	Tags every resource, so the lab is easy to find in the console or in Cost Explorer.

4.2 variables.tf

Code	What it does and why
`variable "region" { default = "us-east-1" }`	The Sophos AMI ID below is specific to this region.

`variable "availability_zone" { default = "us-east-1a" }`	Both sites go in one AZ. That's fine for a lab, but production would spread them.
`variable "admin_cidr" { type = string }`	No default, so you must supply your IP as a /32 in terraform.tfvars. It locks WebAdmin (4444) and SSH (22) to you.
`variable "sophos_ami" { default = "ami-02cfeae4bb63590a2" }`	The SFOS 22.0.2.546 PAYG image, found with `aws ec2 describe-images --owners aws-marketplace --filters Name=name,Values=*ophos*`.
`variable "firewall_instance_type" { default = "t3.medium" }`	The smallest size Sophos's template allows (2 vCPU / 4 GB).
`variable "client_instance_type" { default = "t3.micro" }`	The clients only run curl and ping, so the cheapest size is enough.

4.3 main.tf: sites map and secrets

Code	What it does and why
`locals { sites = { a = { prefix = "10.10", peer = "b" } b = { prefix = "10.20", peer = "a" } } }`	Describes the two sites once. Every resource below uses `for_each = local.sites`, so each one is created twice, once per site. `peer` lets each site look up the other site's public IP for the VPN rules.
`resource "random_id" "suffix" { byte_length = 2 }`	Adds a 4-character hex suffix (for example, `7afa`) to stack names. The template creates Secrets Manager secrets named after the stack, and deleted secrets block that name for 30 days. Without the suffix, a destroy followed by a re-deploy would fail.
`resource "random_password" "admin" { for_each = local.sites length = 20 override_special = "!#%*-_=+" min_upper = 2 min_lower = 2 min_numeric = 2 min_special = 2 }`	A unique 20-character admin password per firewall. The minimums satisfy the template's password rule (upper, lower, digit and special characters, at least 10 long). The special characters are limited to ones that are safe in JSON and the shell.
`resource "random_password" "config_backup" {...}`	A separate password that encrypts configuration backups. Keeping it separate from the admin password limits the damage if one leaks.
`resource "tls_private_key" "lab" { algorithm = "RSA" rsa_bits = 4096 } resource "aws_key_pair" "lab" { public_key = ... } resource "local_sensitive_file" "ssh_key" { file_permission = "0600" }`	Generates an SSH key and uploads the public half to AWS, so the firewall's admin account and the Ubuntu clients trust it. The private half is saved locally with 0600 permissions (SSH refuses keys that other users can read) and is git-ignored.

4.4 main.tf: the firewalls (CloudFormation stack per site)

Code	What it does and why

`resource "aws_cloudformation_stack" "fw" {`
`  for_each = local.sites`
`  name = "sophos-lab-site-${each.key}-${random_id.suffix.hex}"`
`  template_body = file(".../sophos-standalone-payg.yaml")`
`  capabilities = ["CAPABILITY_IAM"]`
	Deploys Sophos's own official template (from github.com/sophos-iaas/aws-cf-templates) once per site. Using the vendor template means the firewall's first-boot setup (passwords, EULA, interfaces) is done exactly the way Sophos supports. `CAPABILITY_IAM` is needed because the template creates an IAM role that lets the firewall read its password from Secrets Manager.

`  AMI = var.sophos_ami`
`  AvailabilityZone = var.availability_zone`
`  PublicNetworkPrefix = each.value.prefix`
	The template builds a new VPC `<prefix>.0.0/16`, with a private LAN `<prefix>.0.0/24` and a public WAN `<prefix>.1.0/24`.

`  InstanceSize = var.firewall_instance_type` `  KeyName = aws_key_pair.lab.key_name`	The instance size, and the SSH key the firewall's admin console will trust.
`  TrustedNetworkCIDR = var.admin_cidr` `  PublicNetworkCIDR  = var.admin_cidr`	The template creates two security groups. "Trusted" allows all TCP, including 22 and 4444, and "Public" allows everything except those two. Setting both to your /32 means nothing else on the internet can reach the firewall over TCP.
`  OptUsingEIPonFirewall = "yes"`	Allocates an Elastic IP: a static public address that survives reboots, which the VPN peer needs.
`  BasicAdminPassword   = random_password.admin[each.key].result` `  ConfigBackupPassword = random_password.config_backup[each.key].result`	The template stores these in Secrets Manager. The firewall reads them on first boot through user-data, so no password is ever typed in by hand.
`  AgreeUserTerms = "yes"  SendLearningStats = "off"` `  CentralUsername = ""  EnableCloudWatch = "no"`	Accepts the EULA, which the template requires. Opts out of telemetry. Skips Sophos Central registration and CloudWatch logging, which would add cost.
`  lifecycle { ignore_changes = [ parameters["BasicAdminPassword"], ... ] }`	CloudFormation masks "NoEcho" parameters as `****` when Terraform reads them back. Without this line, every plan sees a fake password change and updates the stack (this was a real problem, see section 13).

What the template creates per site: a VPC; public and private subnets; an Internet Gateway; route tables, with the private subnet's `0.0.0.0/0` pointing at the firewall's LAN interface; two network interfaces with **source/destination check disabled** (required for any EC2 instance that routes traffic for others); three security groups; an Elastic IP; secrets; an IAM role; and the firewall instance itself (interface 0 = PortA/LAN, interface 1 = PortB/WAN).

4.5 main.tf: looking up what the template built

Code	What it does and why
`data "aws_subnet" "lan" { filter { name = "tag:aws:cloudformation:stack-name" values = [stack name] } filter { name = "tag:aws:cloudformation:logical-id" values = ["SubnetPrivate"] } }`	The template's outputs don't include subnet, VPC or security group IDs. CloudFormation automatically tags everything it creates with the stack name and the resource's logical ID, so Terraform can find the private subnet by those tags. The same trick finds the VPC, the WAN security group, and the LAN interface.
`locals { fw_public_ip = { for k, s in aws_cloudformation_stack.fw : k => s.outputs["PublicIPAddress"] } }`	A map of site → Elastic IP, used by the VPN security group rules.

4.6 main.tf: opening IPsec between the two firewalls only

Code	What it does and why
`resource "aws_vpc_security_group_ingress_rule" "ike" { ip_protocol = "udp" from_port = 500 to_port = 500 cidr_ipv4 = "${peer EIP}/32" }`	IKE (Internet Key Exchange) negotiates the tunnel on UDP 500. The Sophos template only opens TCP, so without this the tunnel can never come up. Only the peer firewall's IP is allowed.
`resource "...ingress_rule" "nat_t" { udp 4500 }`	NAT Traversal. Both firewalls sit behind AWS's 1:1 NAT (private IP ↔ Elastic IP). IKE detects the NAT and moves to UDP 4500, wrapping ESP inside UDP so the NAT can track it.
`resource "...ingress_rule" "esp" { ip_protocol = "50" }`	ESP (IP protocol 50) is raw encrypted traffic, used when there's no NAT. It's included for completeness and doesn't hurt.

4.7 main.tf: clients and access to them

Code	What it does and why
`data "aws_ami" "ubuntu" { owners = ["099720109477"] values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"] }`	The latest official Ubuntu 24.04 image. 099720109477 is Canonical's AWS account, so it can't be a look-alike image from someone else.
`resource "aws_security_group" "eice" { egress tcp 22 → LAN /24 }`	The security group for the EC2 Instance Connect Endpoint. It can only open SSH to the LAN.
`resource "aws_security_group" "client" { ingress 22 from eice SG ingress all from 10.10.0.0/16, 10.20.0.0/16 egress all }`	The clients accept SSH only from the endpoint, plus anything from the two lab networks (the local LAN and the remote site over the tunnel). Filtering is the firewall's job. This group only keeps out the rest of the internet.

`resource "aws_ec2_instance_connect_endpoint" "site" { subnet_id = LAN }`	A free AWS service that tunnels SSH to private instances. The clients need no public IP and no bastion host.
`resource "aws_instance" "client" { private_ip = "${prefix}.0.50" associate_public_ip_address = false metadata_options { http_tokens = "required" } }`	A fixed IP, so the tests can target the remote client. No public IP, so the only way in or out is through the firewall. `http_tokens = required` enforces IMDSv2, which blocks SSRF-style theft of instance credentials.

4.8 outputs.tf

This file prints everything you need after the deploy: the WebAdmin URL, the SSH command, the public IP, the WAN and LAN private IPs, the local and peer networks, and the peer's public IP. The passwords are marked `sensitive`, so they only show when you ask for them explicitly (`terraform output -json admin_passwords`).

4.9 .gitignore: what must never reach GitHub

Line	Why
`terraform/*.tfstate  terraform/*.tfstate.*`	The state file contains every generated password in plain text.
`terraform/*.tfvars`	Holds your home IP address.
`terraform/*.tfplan`	Saved plans also contain the secrets.
`terraform/.keys/`	The SSH private key.
`backups/`	Firewall configuration backups.

5. Phase 2: Deploy

```
cd terraform
echo 'admin_cidr = "73.249.174.243/32"' > terraform.tfvars
terraform init       # downloads the aws, random, tls and local providers
terraform validate   # syntax and type check
terraform plan -out=lab.tfplan   # shows the 24 resources to be created
terraform apply lab.tfplan       # builds it (about 15 min, mostly the firewall stacks)
terraform output firewalls       # IPs and URLs
```

Command	Explanation
`terraform init`	Downloads the provider plugins and creates the `.terraform/` folder and the lock file (exact provider versions, committed so builds can be repeated).
`terraform plan -out`	Compares the code with reality and saves the exact change set. Applying a saved plan guarantees that what you reviewed is what runs.
`terraform apply`	Creates the resources in dependency order: keys and passwords, then the two stacks in parallel, then the lookups, security group rules, endpoints and clients.

> Wait about 10 minutes after the apply before opening WebAdmin. EC2 reports "running" well before SFOS has finished booting. Check with `curl -k https://<EIP>:4444`: a 200 response means it's ready.

6. Phase 3: First login

1. Get the passwords: `terraform output -json admin_passwords`.
2. Browse to `https://44.218.174.190:4444` and accept the self-signed certificate warning.
3. **CAPTCHA:** SFOS shows a CAPTCHA for admin logins arriving on the WAN. Because the security group already limits access to one IP, it was turned off from the SSH console:

   ```
   ssh -i .keys/sophos-lab.pem admin@44.218.174.190
     → 4. Device Console
   console> system captcha-authentication-global disable    (answer Y)
   console> system captcha-authentication-global show
   ```

   Turn it back on with `... enable` when you're finished.
4. Dismiss the first-login popups: "Create secure storage master key" → **Skip for now**, and "Sophos Central" → **No thanks**.
5. **Network** → **Interfaces:** confirm PortA = LAN (10.10.0.247) and PortB = WAN (10.10.1.125).
6. **Hosts and services** → **IP host:** create the network objects `LAN-A-Net` (10.10.0.0/24) and `LAN-B-Net` (10.20.0.0/24) on both firewalls.
7. **Hosts and services** → **Services:** create `ICMP-Echo` (type ICMP, Echo Request, any code).

Named objects instead of raw IPs make the rules readable, and a change in one place updates every rule that uses the object.

7. Phase 4: Firewall rules and NAT

Go to **Rules and policies → Firewall rules → Add firewall rule → New firewall rule**. Sophos evaluates rules **top-down, and the first match wins**, so the deny rule must sit above the allow rule.

#	Name	Action	Source	Destination	Service	Other
1	Deny-LAN-ICMP-Internet	Drop	Zone LAN · LAN-A-Net	Zone WAN · Any	ICMP-Echo	Log firewall traffic ✔ · position Top
2	Allow-LAN-to-WAN	Accept	Zone LAN · LAN-A-Net	Zone WAN · Any	Any	Log ✔ · Web policy (Phase 5) · NAT: default SNAT (MASQ)

Line-by-line: what each field means

Field	Meaning
Source zone LAN / networks LAN-A-Net	Matches traffic that enters on the LAN port and comes from 10.10.0.0/24. Using both zone and network is defense in depth against spoofed sources.
Destination zone WAN	Matches traffic that's leaving to the internet. VPN traffic goes to the VPN zone, so these rules don't touch it.
Action Drop vs Reject	Drop discards silently, so the sender times out. Reject sends back an ICMP unreachable or a TCP RST. Drop gives an attacker less information, and Reject fails faster for real users.
Log firewall traffic	Writes every match to the log viewer. You need it for troubleshooting and audit evidence.
NAT (masquerade / MASQ)	Rewrites the source 10.10.0.50 to the PortB address 10.10.1.125, and AWS then maps that to 44.218.174.190. Without it, replies from the internet would have no way back to a private address. Check that the **Default SNAT IPv4** rule exists under **Rules and policies → NAT rules**.

Verify: from client-site-a, `curl https://checkip.amazonaws.com` should return 44.218.174.190, which proves NAT. `ping 1.1.1.1` should fail, and **Log viewer → Firewall** should show a drop entry for rule *Deny-LAN-ICMP-Internet*.

8. Phase 5: Web filtering

1. **Web → Policies → Add policy**, named `Lab-Block-Social`.
2. Add a rule: Users = Anyone · Activities = category **Social networking** · Action = **Block**. Leave the default as allow.

3. Edit firewall rule 2 (*Allow-LAN-to-WAN*) and set **Web policy** = Lab-Block-Social. Leave HTTPS decryption off.

How it blocks HTTPS without decryption: the TLS ClientHello carries the hostname in plain text (SNI, Server Name Indication). Sophos reads the SNI, classifies it, and blocks the connection. The client sees a TLS failure or a Sophos block certificate instead of the real site. With decryption on, Sophos could show a friendly block page, but every client would need the Sophos CA certificate installed.

Verify: `curl -sf https://www.facebook.com` fails, while `curl -sf https://www.wikipedia.org` works. The block shows in **Log viewer** → **Web filter**.

9. Phase 6: Site-to-site IPsec VPN

First, on both firewalls: go to **Administration** → **Device access** and tick **IPsec** in the **WAN** row, then click Apply and OK. Without this, the firewall's local service ACL drops IKE packets before the VPN engine sees them.

Then go to **Site-to-site VPN** → **IPsec** → **Add**. Configure Site B (the responder) first, then Site A (the initiator). After you save, make sure the **Active** dot is green on both sides, and click the **Connection** dot on Site A to bring the tunnel up.

Field	Site A	Site B	Explanation
Name	S2S_A_to_B	S2S_B_to_A	Letters, digits and underscores only. Hyphens are rejected.
Connection type	Site-to-site	Site-to-site	Network-to-network, as opposed to remote access for a single user.
Gateway type	Initiate the connection	Respond only	Exactly one side starts IKE. Two initiators can collide and create duplicate SAs.
Activate on save / Create firewall rule	✔ / ✔	✔ / ✔	Sophos creates the LAN ↔ VPN allow rule automatically.
Encryption profile	IKEv2 (same on both)	IKEv2	Phase 1 (IKE SA) and phase 2 (IPsec SA) algorithms. They must match exactly on both sides, or the negotiation fails with "no proposal chosen".
Authentication	Preshared key	Same key	Generate it with `openssl rand -base64 32`. Certificates would be the production upgrade.
Listening interface	PortB 10.10.1.125	PortB 10.20.1.211	The WAN port. The firewall only knows its private IP.
Local ID type / ID	IP · 44.218.174.190	IP · 23.21.5.108	**The key AWS detail.** By default, the ID would be the private 10.x address, which the peer doesn't expect. Each side must present its public EIP as its identity.
Local subnet	LAN-A-Net	LAN-B-Net	What this side protects (the traffic selector).
Remote gateway	23.21.5.108	44.218.174.190	The peer's public IP.
Remote ID type / ID	IP · 23.21.5.108	IP · 44.218.174.190	Must equal the peer's Local ID.
Remote subnet	LAN-B-Net	LAN-A-Net	Must mirror the peer's local subnet, or phase 2 fails ("TS_UNACCEPTABLE").

What happens on the wire

1. **IKE_SA_INIT** (UDP 500): the two sides agree on encryption and exchange Diffie-Hellman keys. Each side's NAT-detection hashes reveal the AWS NAT, so both move to UDP 4500.
2. **IKE_AUTH** (UDP 4500): each side proves its identity with the PSK and its ID. The first CHILD_SA, the IPsec tunnel for 10.10.0.0/24 ↔ 10.20.0.0/24, is created.
3. Data flows as ESP inside UDP 4500. Keys are re-negotiated before they expire (rekeying), and dead-peer detection notices if the other side disappears.

Verify: the IPsec connection shows green on both sides. From client-site-a, run `ping 10.20.0.50`, and on client-site-b run `sudo tcpdump -ni ens5 icmp`. The packets arrive with source 10.10.0.50, which proves there's no NAT across the tunnel. On fw-a, **Diagnostics** → **Packet capture** with the filter `host 10.20.0.50` shows the cleartext entering the tunnel, while a capture on PortB shows only UDP 4500 to 23.21.5.108. That's the proof that the traffic is encrypted on the wire.

10. Phase 7: Validation script (line by line)

Code	What it does and why
`SITE="${1:?usage: client-tests.sh a|b}"`	Requires the site letter, and exits with the usage message if it's missing.
`case "$SITE" in a) PEER=10.20.0.50 ;; b) PEER=10.10.0.50 ;; esac`	Each site tests the other site's client.
`exec > >(tee "$OUT") 2>&1`	Sends all output to the screen and also to a timestamped results file, which becomes your evidence.
`check() { ... timeout 15 "$@" ... }`	Runs a command with a 15-second limit and compares the result to what's expected ("ok" or "blocked"). A blocked test that fails counts as a PASS.
`check "HTTPS to internet" ok curl -sS -o /dev/null https://www.example.com`	Proves the allow rule and NAT work.
`check "DNS resolution" ok getent hosts www.amazon.com`	Proves DNS works through the firewall.
`curl -s https://checkip.amazonaws.com`	Prints the public IP the internet sees. It must be the site's EIP.
`check "ICMP ping to 1.1.1.1" blocked ping -c 2 -W 3 1.1.1.1`	Proves the deny rule works and comes before the allow rule.
`check "Social-networking site" blocked curl -sf https://www.facebook.com`	Proves web filtering. `-f` makes curl fail on errors or a block.
`check "Allowed category site" ok curl -sf https://www.wikipedia.org`	A control test: proves the filter is selective, not blocking everything.

`check "Ping remote site client" ok ping -c 3 -W 3 "$PEER"`	Proves the IPsec tunnel carries ICMP.
`check "TCP 22 to remote" ok bash -c "exec 3<>/dev/tcp/$PEER/22"`	Proves the tunnel carries TCP, using bash's built-in socket instead of installing netcat.

To run it, copy the script to the client (`terraform output clients` prints the SSH command), then run `bash client-tests.sh a`.

11. Phase 8: Backup and restore

1. One-time setup: create the **secure storage master key** (from the dashboard prompt, or under Backup & firmware). SFOS v22 won't create backups without it.
2. **Backup & firmware** → **Backup & restore**: choose Local, set the encryption password, click **Backup now**, then **Download** → **Download backup**. The file is encrypted.
3. Make a visible change: disable rule 1 *Deny-LAN-ICMP-Internet*. `ping 1.1.1.1` from the client now **works**.
4. **Upload and restore**: choose the file and enter the backup password, click OK on the warning, enter the **SSMK**, then confirm the **interface mapping** (PortA → PortA, PortB → PortB) and click Restore. The firewall reboots, which took about 6 minutes in this lab.
5. Verify: rule 1 is enabled again, and `ping 1.1.1.1` fails again. Re-run `client-tests.sh a` to capture the evidence.

Why this matters: a backup that has never been restored is only a hope. Proving the restore, and that it reverts a known change, is what change management means by a "tested rollback plan".

12. Phase 9: Teardown and cost control

```
cd terraform && terraform destroy
```

This removes the clients, endpoints and security group rules, then both CloudFormation stacks (firewalls, VPCs, EIPs, secrets and IAM roles). Afterwards, check with `aws ec2 describe-instances --filters Name=tag:Project,Values=sophos-s2s-lab`. Cancel the Marketplace subscription if you won't rebuild. The lab costs roughly $0.10–0.15/hour in EC2, plus the Sophos PAYG software fee per firewall, so destroy it every time you finish.

Results: what the real build produced

Every result below was captured from the live lab on 14 October 2026.

Automated tests: 14 of 14 passed (after the backup restore and reboot)

```
PASS  HTTPS to internet (LAN->WAN allow + NAT)               (expected ok)
PASS  DNS resolution                                         (expected ok)
      public source IP seen by internet: 44.218.174.190
PASS  ICMP ping to 1.1.1.1 (deny rule)                       (expected blocked)
PASS  Social-networking site (web filter block)              (expected blocked)
PASS  Allowed category site still works                      (expected ok)
PASS  Ping remote site client 10.20.0.50 over IPsec          (expected ok)
PASS  TCP 22 to remote site client 10.20.0.50 over IPsec     (expected ok)
Result: 7 passed, 0 failed  (saved to results-site-a-20261014-200504.txt)

PASS  HTTPS to internet (LAN->WAN allow + NAT)               (expected ok)
PASS  DNS resolution                                         (expected ok)
      public source IP seen by internet: 23.21.5.108
PASS  ICMP ping to 1.1.1.1 (deny rule)                       (expected blocked)
PASS  Social-networking site (web filter block)              (expected blocked)
PASS  Allowed category site still works                      (expected ok)
PASS  Ping remote site client 10.10.0.50 over IPsec          (expected ok)
PASS  TCP 22 to remote site client 10.10.0.50 over IPsec     (expected ok)
Result: 7 passed, 0 failed  (saved to results-site-b-20261014-200513.txt)
```

IPsec security association (fw-site-a, `ipsec statusall`)

```
Connections:
S2S_A_to_B-1:  10.10.1.125...23.21.5.108  IKEv2, dpddelay=30s
S2S_A_to_B-1:   local:  [44.218.174.190] uses pre-shared key authentication
S2S_A_to_B-1:   remote: [23.21.5.108] uses pre-shared key authentication
S2S_A_to_B-1:   child:  10.10.0.0/24 === 10.20.0.0/24 TUNNEL, dpdaction=start
Security Associations (1 up, 0 connecting):
S2S_A_to_B-1[1]: ESTABLISHED 3 minutes ago,
10.10.1.125[44.218.174.190]...23.21.5.108[23.21.5.108]
S2S_A_to_B-1[1]: IKEv2 SPIs: a8946b3264315c8c_i* 85f6f3e65487f29b_r, rekeying in 78 minutes
S2S_A_to_B-1[1]: IKE proposal: AES_CBC_256/HMAC_SHA2_512_256/PRF_HMAC_SHA2_512/CURVE_25519
S2S_A_to_B-1{1}:  INSTALLED, TUNNEL, reqid 1, ESP in UDP SPIs: c9b3b081_i cd01b9f5_o
S2S_A_to_B-1{1}:  AES_CBC_256/HMAC_SHA2_512_256, 1259 bytes_i (18 pkts, 24s ago), 1259 bytes_o
(18 pkts, 24s ago), rekeying in 46 minutes
S2S_A_to_B-1{1}:   10.10.0.0/24 === 10.20.0.0/24
```

How to read it: **ESTABLISHED** means IKE completed. The **local [44.218.174.190] / remote [23.21.5.108]** IDs are the Elastic IPs, as configured. **ESP in UDP** confirms NAT-T on 4500. The ciphers are AES-256-CBC with SHA2-512 and Curve25519 Diffie-Hellman, and the selectors are **10.10.0.0/24 === 10.20.0.0/24**.

No NAT across the tunnel (tcpdump on client-site-b while client-site-a pings it)

```
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens5, link-type EN10MB (Ethernet), snapshot length 262144 bytes
19:11:04.801477 IP 10.10.0.50 > 10.20.0.50: ICMP echo request, id 1800, seq 1, length 64
19:11:04.801509 IP 10.20.0.50 > 10.10.0.50: ICMP echo reply, id 1800, seq 1, length 64
19:11:05.802758 IP 10.10.0.50 > 10.20.0.50: ICMP echo request, id 1800, seq 2, length 64
19:11:05.802785 IP 10.20.0.50 > 10.10.0.50: ICMP echo reply, id 1800, seq 2, length 64
19:11:06.804186 IP 10.10.0.50 > 10.20.0.50: ICMP echo request, id 1800, seq 3, length 64
19:11:06.804213 IP 10.20.0.50 > 10.10.0.50: ICMP echo reply, id 1800, seq 3, length 64
6 packets captured
6 packets received by filter
0 packets dropped by kernel
```

Backup and restore proof

```


--- 1.1.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 2.140/2.168/2.189/0.020 ms

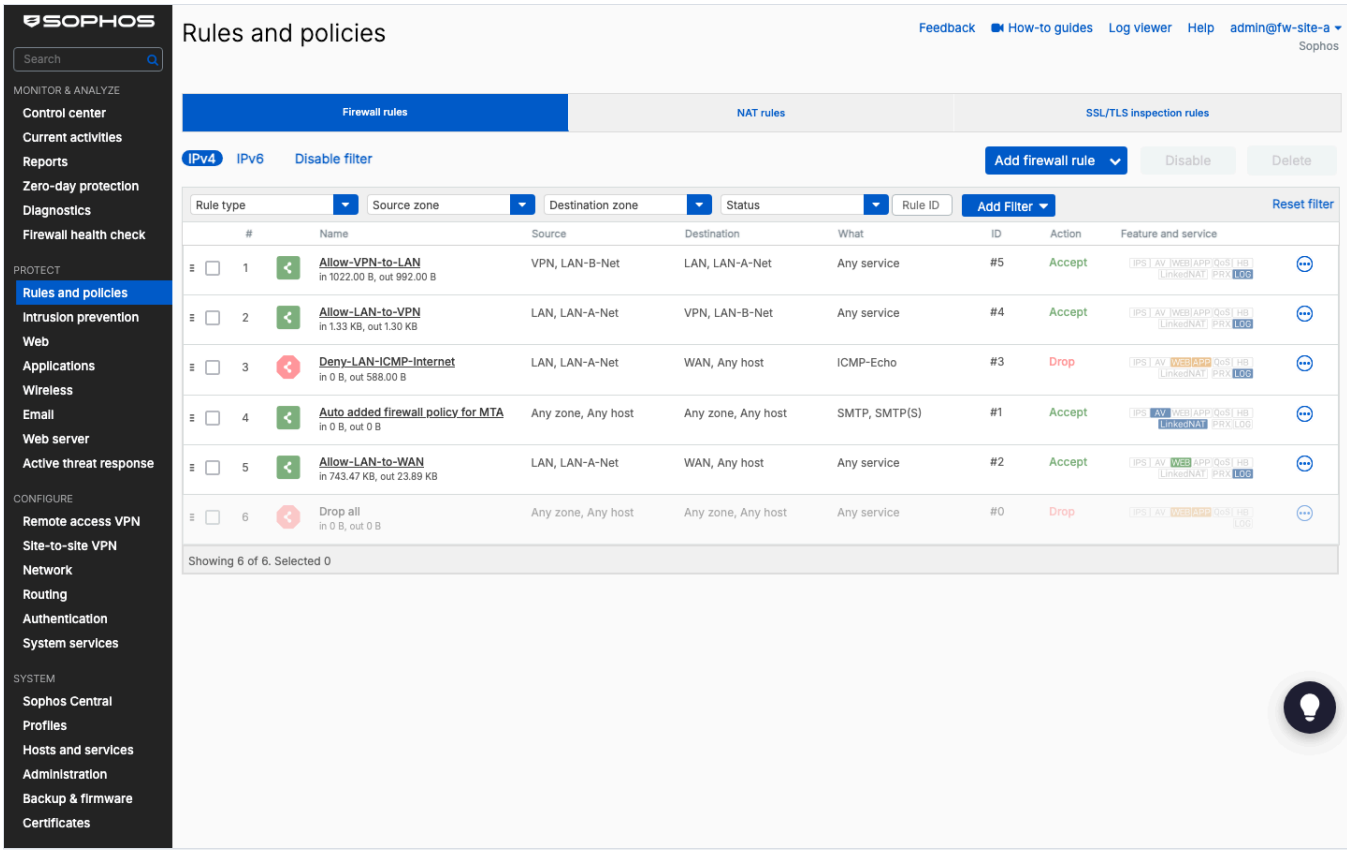

--- 1.1.1.1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2070ms


```

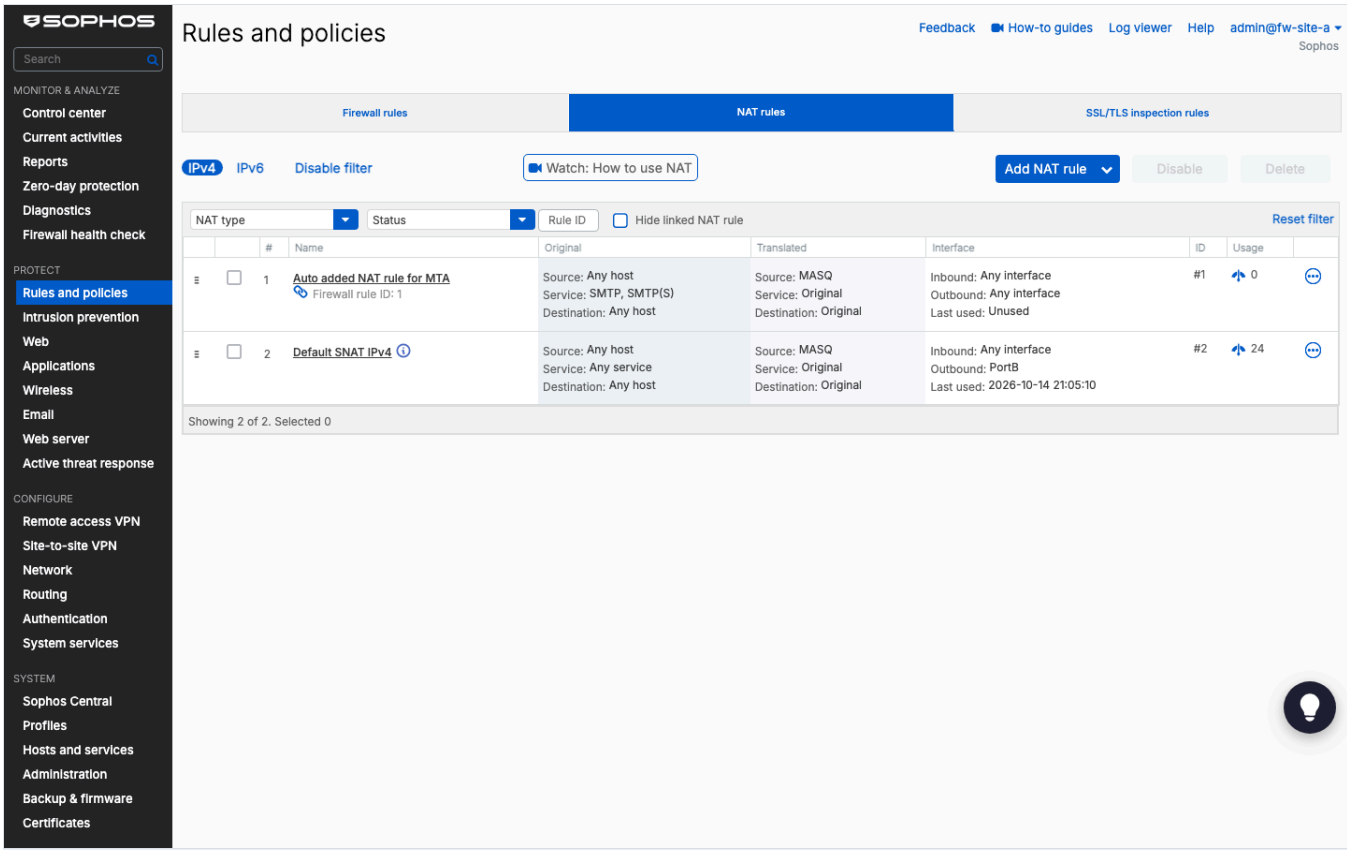
With the deny rule disabled, ping succeeded (0% loss). After the backup was restored, the firewall rebooted, the rule came back **Enabled**, and ping was blocked again (100% loss).

Screenshots

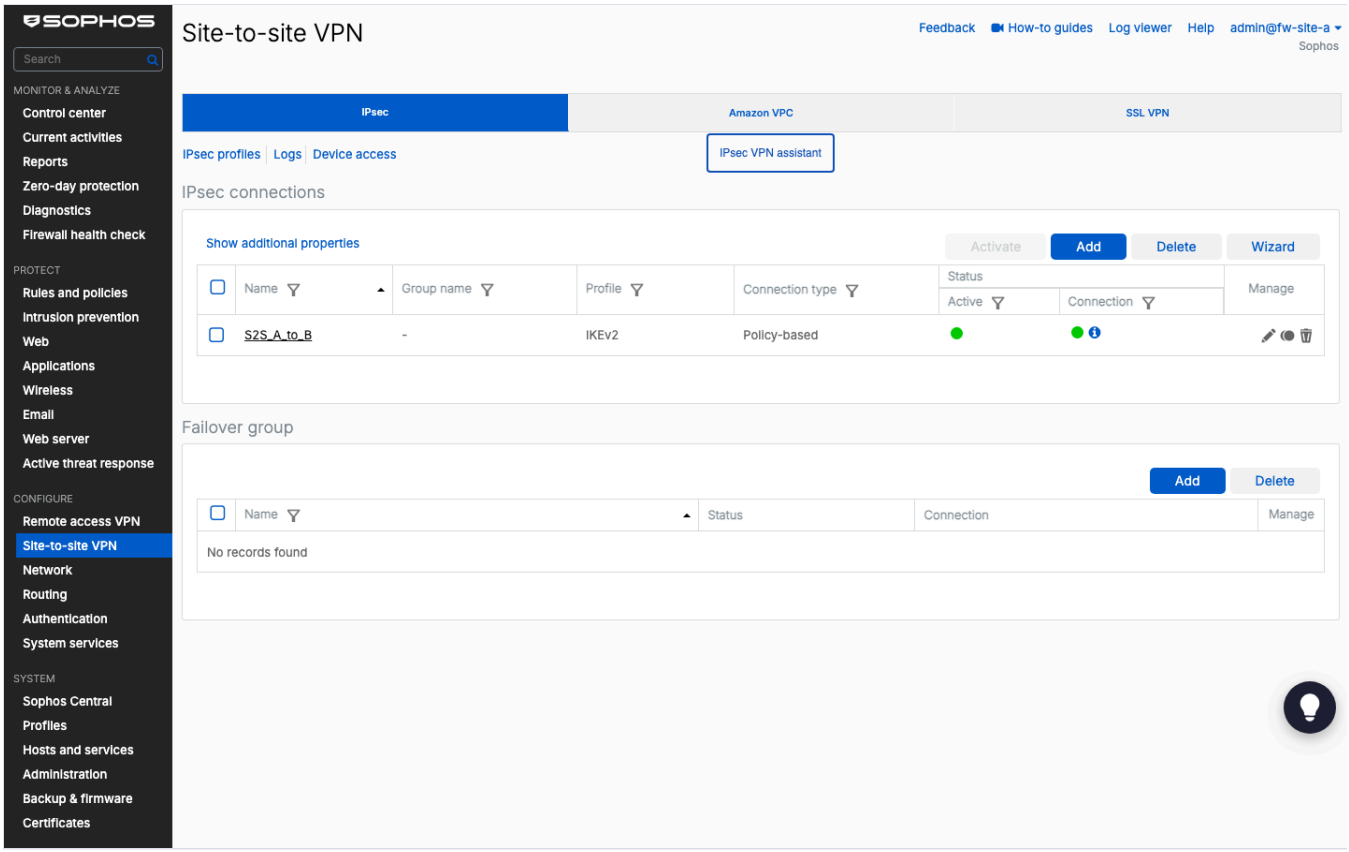
01-site-a-firewall-rules



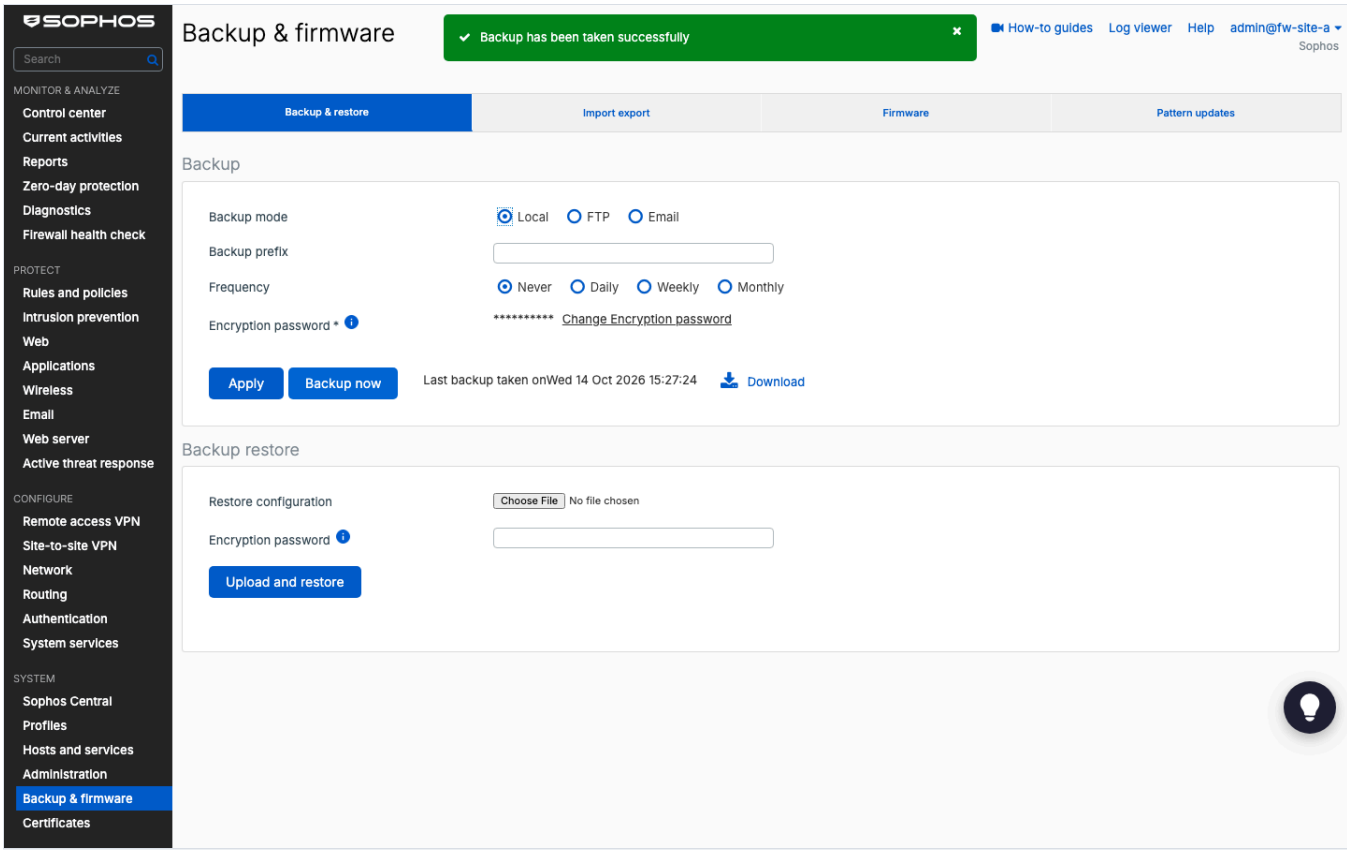
02-site-a-nat-rules



03-site-a-ipsec-connected



04-site-a-backup


SOPHOS
Search
MONITOR & ANALYZE
Control center
Current activities
Reports
Zero-day protection
Diagnostics
Firewall health check
PROTECT
Rules and policies
Intrusion prevention
Web
Applications
Wireless
Email
Web server
Active threat response
CONFIGURE
Remote access VPN
Site-to-site VPN
Network
Routing
Authentication
System services
SYSTEM
Sophos Central
Profiles
Hosts and services
Administration
Backup & firmware
Certificates
Backup & firmware
Backup has been taken successfully
How-to guides
Log viewer
Help
admin@fw-site-a
Sophos
Backup & restore
Import export
Firmware
Pattern updates
Backup
Backup mode
Local
FTP
Email
Backup prefix
Frequency
Never
Daily
Weekly
Monthly
Encryption password *
********** Change Encryption password
Apply
Backup now
Last backup taken onWed 14 Oct 2026 15:27:24
Download
Backup restore
Restore configuration
Choose File
No file chosen
Encryption password
Upload and restore

13. Problems hit during the real build, and the fixes

These came up in the actual build. They're the best material for interviews, because they show real troubleshooting.

Problem	Root cause	Fix
The Sophos ISO won't run on a local VM	The Mac is ARM (Apple M4), and Sophos is x86-64 only.	Moved the lab to AWS EC2 (x86) using the official Marketplace AMI.
The launch would fail with `OptInRequired`	Marketplace AMIs need the product's terms accepted first.	Subscribed to the PAYG listing, and checked with a dry-run before deploying.
The tunnel could never come up (found in design review)	The vendor template only opens TCP on the WAN, and IPsec needs UDP 500/4500 and ESP.	Added peer-only security group rules in Terraform.
`InvalidParameterValue: Invalid security group description`	The description contained an apostrophe, which AWS doesn't allow.	Reworded the description and re-applied.
The re-apply "changed" the stacks and replaced 10 resources	CloudFormation returns NoEcho password parameters masked, so Terraform saw a change every time. That made the dependent lookups "unknown" and forced the endpoints and rules to be replaced.	Added `lifecycle.ignore_changes` for the password parameters. Confirmed the firewall instances were not replaced (same launch time), and the plan is now "No changes".
WebAdmin login demanded a CAPTCHA	SFOS requires a CAPTCHA for admin logins coming from the WAN.	`system captcha-authentication-global disable` from the device console, safe because the security group allows only one /32. It gets turned back on afterwards.
The IPsec connection was rejected: "Configuration parameters validation failed" (501)	The API response named `/VPNIPSecConnection/Configuration/Name`: SFOS connection names can't contain hyphens.	Renamed to `S2S_A_to_B` / `S2S_B_to_A`.

The tunnel was configured, but `ipsec statusall` showed no connections and cross-site tests failed (5/7)	(1) IPsec wasn't allowed from the WAN zone under **Administration → Device access**, so the firewall dropped IKE before the VPN engine saw it. (2) A connection created through the API is saved but not activated.	Ticked IPsec for the WAN zone on both firewalls, activated both connections, and connected from the initiator. The SA was ESTABLISHED and 7/7 tests passed on both sites.
"To generate the backup, you must create the secure storage master key"	SFOS v22 requires a secure storage master key (SSMK) before it creates backups, so the passwords inside the backup are protected.	Created the SSMK and stored it securely. The restore then asked for the backup password, the SSMK, and an interface-mapping confirmation.
The web filter test failed right after the restore and reboot	The web-filtering service and its category database were still starting.	Re-tested about 2 minutes later and it passed. Lesson: wait for all services to come up before you validate after a reboot.
The XML API returned "You need to enable the API Configuration" (code 532)	API access is off by default and needs an allowed-IP list.	**Administration → API access**: turn it on and add the admin IP host.

General troubleshooting order for a tunnel that won't come up

1. Can the peers reach each other? Check the security groups and NACLs for UDP 500/4500, from the peer's EIP.
2. Phase 1 fails? Check that the IKE version and proposals match, that the PSK is the same, and that the **IDs match** (the classic AWS mistake is a private-IP ID).
3. Phase 2 fails? Check that the traffic selectors mirror each other (local = the peer's remote) and that the PFS/DH groups match.
4. Tunnel up but no traffic? Check the LAN ↔ VPN firewall rules, the AWS route table (default route → the firewall's LAN interface), that source/dest check is off, that NAT isn't being applied to VPN traffic, and the client security groups.
5. Logs: **Log viewer → IPsec** on SFOS, and packet capture on PortB for UDP 500/4500.

14. Interview follow-up questions and answers

1. Walk me through this project.
I built two simulated offices in AWS, each a VPC with a Sophos Firewall and an Ubuntu host, all deployed with Terraform. On the firewalls, I configured zones, NAT, an ordered deny and allow rule set, and category web filtering. Then I connected the sites with an IKEv2 IPsec tunnel. I proved every policy with a PASS/FAIL test script, tcpdump and the firewall logs, and tested a configuration backup and restore.

2. Why did you deploy in AWS instead of on a local VM?
My laptop is Apple Silicon, and Sophos Firewall is x86-only, so it couldn't boot locally. AWS gave me x86, an official Sophos image, and real public IPs. It also added realistic cloud problems like 1:1 NAT and security groups.

3. What's the difference between IKE phase 1 and phase 2?
Phase 1 builds a secure, authenticated channel between the two gateways (the IKE SA): they negotiate algorithms, do a Diffie-Hellman exchange, and authenticate with the PSK. Phase 2 uses that channel to create the IPsec SAs that actually encrypt user traffic, for specific subnets (the traffic selectors). In IKEv2, these map to IKE_SA_INIT/IKE_AUTH and CHILD_SA.

4. Why IKEv2 instead of IKEv1?
IKEv2 needs fewer messages to set up, has built-in NAT traversal and dead-peer detection, handles rekeying more reliably, and is the current standard. IKEv1's aggressive mode with a PSK is also weak against offline cracking.

5. What's NAT-T and why did you need it?
ESP is IP protocol 50 and has no ports, so NAT devices can't track it. NAT-T wraps ESP in UDP 4500. In AWS, each firewall only has a private IP that's mapped 1:1 to an Elastic IP. IKE detects that NAT and switches to UDP 4500, which is why I opened UDP 500 and 4500 between the peers.

6. Why did you set the Local ID to the public IP?
By default, the firewall identifies itself with its interface address, which is a private 10.x IP. The peer is configured to expect the public EIP, so authentication would fail with an ID mismatch. Setting each side's Local ID to its own EIP, and the Remote ID to the peer's EIP, fixes that. It's one of the most common AWS VPN mistakes.

7. Why is rule order important?
Sophos evaluates rules top-down, and the first match wins. If the broad "allow LAN to WAN" rule came first, ICMP would match it and the deny rule would never be reached. Specific deny rules go above broad allow rules.

8. Drop or Reject: which, and when?
Drop discards silently, which gives scanners less information and is right for the WAN side. Reject returns a TCP RST or ICMP unreachable, so internal users and applications fail fast instead of timing out. For internal deny rules, Reject is often friendlier.

9. How does masquerade NAT differ from static NAT?

Masquerade (many-to-one, a form of source NAT) rewrites many private sources to the outgoing interface's IP and uses ports to tell the connections apart. Static NAT maps one private IP to one public IP permanently, which is what AWS does between the firewall's WAN IP and the Elastic IP. Destination NAT (port forwarding) publishes an internal server to the internet.

10. Why isn't traffic NATed across the tunnel?

Both sides route each other's real subnets, so NAT isn't needed, and keeping the real source IPs makes logging and access control meaningful. On the far side, tcpdump showed the source 10.10.0.50, which proved it. NAT inside a tunnel is only used when the subnets overlap.

11. How does web filtering block HTTPS without decrypting it?

The TLS ClientHello includes the hostname in clear text (SNI). The firewall reads the SNI, looks up its category, and blocks the handshake. Full decryption allows a block page and deeper inspection, but every client needs the firewall's CA certificate installed, and it raises privacy and compatibility concerns.

12. How did you verify the traffic was really encrypted?

I compared two captures. On the LAN side, I could see the cleartext ICMP between 10.10.0.50 and 10.20.0.50. On the WAN port, the only traffic was UDP 4500 between the two Elastic IPs, with no inner IPs visible. The remote client's tcpdump showed the packets arriving decrypted.

13. What AWS settings make an EC2 instance work as a router?

Source/destination check has to be disabled on its interfaces, because otherwise AWS drops packets that aren't addressed to the instance itself. The private subnet's route table needs `0.0.0.0/0` pointing at the firewall's LAN interface, and the security groups and NACLs have to allow the traffic.

14. How did you secure management access?

WebAdmin (4444) and SSH (22) are only reachable from my /32, through the security groups. The clients have no public IPs and are reached only through EC2 Instance Connect Endpoints. The passwords are randomly generated and stored in Secrets Manager. IMDSv2 is enforced on the clients, and the IPsec ports are open only between the two peers.

15. If the tunnel was down, how would you troubleshoot it?

I'd work bottom-up. First, reachability (security groups for UDP 500/4500). Then phase 1: proposals, PSK, and IDs. Then phase 2: the subnets mirror each other and PFS matches. Then the data path: firewall rules, routes, source/dest check, and no NAT on VPN traffic. Throughout, I'd use the IPsec log and a packet capture on the WAN port.

16. Why test the backup by restoring it?

A backup that has never been restored is unproven. I took a backup, made a known change by disabling the deny rule, restored the backup, and confirmed the change was gone with a failed ping. That proves both the backup and the rollback procedure.

17. Why Terraform plus a CloudFormation template, instead of all Terraform?

Sophos's template contains the vendor-supported first-boot integration: the Secrets Manager password handoff, the EULA, and the interface order. Wrapping it in Terraform kept that support and still let me manage everything else, like the clients, the VPN security group rules and the endpoints, with one tool and one state file.

18. What was a real problem you hit, and how did you solve it?

After a re-apply, Terraform reported that the firewall stacks had changed, and it replaced ten dependent resources. I traced it to CloudFormation masking NoEcho password parameters, so every plan saw a fake diff. I confirmed the firewalls themselves hadn't been replaced, added `ignore_changes` for those parameters, and verified that the plan was clean.

19. PSK or certificates in production?

Certificates. A PSK is a shared secret that has to be distributed and rotated by hand, and a weak one can be brute-forced. Certificates give each gateway its own identity, can be revoked, and scale to many sites.

20. How would you make this production-ready?

High availability (two firewalls with failover or a Gateway Load Balancer design), certificate authentication, IPS turned on, centralized logging to a SIEM, Sophos Central management, scheduled encrypted backups to S3, and multiple Availability Zones. I'd also keep the Terraform state in a remote encrypted backend with locking.

21. Policy-based or route-based VPN: what's the difference?

A policy-based VPN encrypts traffic that matches defined source and destination subnets (what this lab uses). A route-based VPN creates a virtual tunnel interface (XFRM in Sophos), and the routing table decides what enters it. That's more flexible, and it supports dynamic routing (BGP) and many subnets.

22. What do the firewall logs show for a dropped packet?

The time, the rule ID and name that matched, the action (drop), the in and out interfaces and zones, the source and destination IPs and ports, and the protocol. That's enough to prove which rule acted and to tune it.

15. STAR stories

Story 1: Building a secure two-site network end to end

Situation I wanted hands-on firewall and VPN experience that matched entry-level network security job postings: firewall rules, NAT, VPN and configuration backups.

Task Build a realistic two-office network protected by enterprise firewalls, connect the offices securely, and prove each control works.

Action I wrote Terraform to deploy two Sophos Firewalls in separate AWS VPCs, with private client subnets routed through them. I configured NAT, an ordered deny and allow rule set with logging, and category web filtering. I set up an IKEv2 IPsec tunnel, handling AWS 1:1 NAT with NAT-T and public-IP identities. Then I wrote a test script that checks every policy.

Result Every test passed: internet through NAT, ICMP blocked and logged, social media blocked while other sites worked, and cross-site traffic flowing over the tunnel without NAT. I can rebuild the whole infrastructure with one command and tear it down to control cost.

Story 2: Troubleshooting an unexpected infrastructure change

Situation After a minor fix, Terraform reported that both firewall stacks had "changed", and it replaced ten dependent resources.

Task Find out whether the firewalls had been rebuilt, and stop it from happening again.

Action I read the plan diff and saw that the only "changes" were masked password parameters. I checked the instance IDs and launch times to confirm the firewalls had not been replaced. Then I added `lifecycle.ignore_changes` for those NoEcho parameters and re-ran the plan.

Result The plan came back "No changes", and the firewalls and their configuration were untouched. I learned to always read plan diffs before applying, and how CloudFormation masks sensitive parameters.

Story 3: Making a site-to-site VPN work through cloud NAT

Situation Each firewall in AWS only knows its private WAN IP. AWS maps it 1:1 to a public Elastic IP, and the vendor template didn't allow IPsec traffic.

Task Get an IKEv2 tunnel to come up reliably between the two sites.

Action I opened UDP 500 and 4500 and ESP in the security groups, only between the two peer IPs. I set each firewall's Local ID to its own public IP and the Remote ID to the peer's, made sure the traffic selectors mirrored each other, and made one side initiator and the other responder.

Result The tunnel came up using NAT-T. A packet capture showed only UDP 4500 on the WAN, while the remote host received the decrypted traffic with the original source IP.

Story 4: Proving a rollback plan works

Situation The job posting I was targeting listed "backup and restore network device configurations" as a duty.

Task Show that I could back up and restore a firewall configuration, and verify it.

Action I took an encrypted configuration backup and made a deliberate change (disabling the ICMP deny rule), confirming that the change took effect. Then I restored the backup and re-ran my tests.

Result After the restore, the deny rule was back and ping was blocked again, which proved that the backup and the rollback procedure both work.

16. Resume entry

Sophos Firewall Site-to-Site VPN Lab (AWS) | github.com/Jeevanandh32/sophos-firewall-site-to-site-vpn-lab

- Provisioned two Sophos Firewall (SFOS v22) sites in separate AWS VPCs with Terraform, each with WAN/LAN subnets and an Ubuntu host routed through the firewall.
- Built an IKEv2 IPsec site-to-site VPN (10.10.0.0/24 ↔ 10.20.0.0/24), working through AWS 1:1 NAT with NAT-T and Elastic IP-based IKE identities.
- Configured NAT, ordered allow and deny firewall rules with logging, and category web filtering, and validated them with automated tests, tcpdump, packet capture and firewall logs.
- Tested configuration backup and restore, and reproduced the infrastructure with a single `terraform apply`.

Skills: Sophos Firewall · IPsec/IKEv2 · NAT · Firewall policy · Web filtering · AWS (VPC, EC2, Security Groups, CloudFormation) · Terraform · tcpdump · Linux

> Keep the resume claims accurate. "Rebuilt the full lab with one Terraform command" is true for the infrastructure. The firewall policy itself comes back from the configuration backup, not from Terraform. Say it that way if you're asked, because interviewers respect precision.